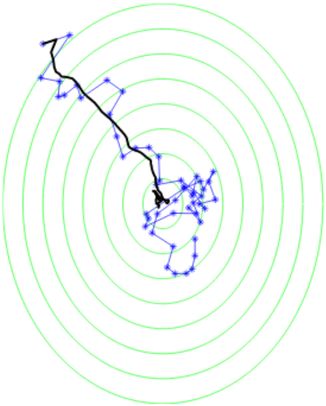
Optimization in Machine Learning

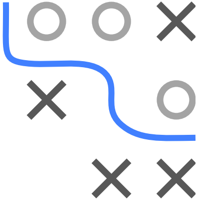
First order methods
SGD



Learning goals

- Understand stochastic gradient descent
- Understand stochasticity and variance of SGD
- Understand effect of batch size and step size
- Understand stopping rules

STOCHASTIC GRADIENT DESCENT



- $g : \mathbb{R}^d \to \mathbb{R}$ objective, g is **average over functions**:

$$g(\boldsymbol{x}) = \frac{1}{n}\sum_{i=1}^{n} g_i(\boldsymbol{x}), \qquad g \text{ and } g_i \text{ smooth}$$

NB: We use g instead of f as objective, because f is used as model in ML

- Stochastic gradient descent (SGD) approximates the gradient

$$\nabla_{\boldsymbol{x}} g(\boldsymbol{x}) = \frac{1}{n}\sum_{i=1}^{n} \nabla_{\boldsymbol{x}} g_i(\boldsymbol{x}) \quad := \quad \mathbf{d} \quad \text{by}$$

$$\frac{1}{|J|}\sum_{i \in J} \nabla_{\boldsymbol{x}} g_i(\boldsymbol{x}) \quad := \quad \hat{\mathbf{d}},$$

with random subset $J \subset \{1, 2, \dots, n\}$ called **mini-batch**

- This is done e.g. when computing the true gradient is **expensive**
- SGD is a **stochastic optimization** method as it uses stochastic gradient estimates

SGD ALGORITHM

Algorithm Basic SGD pseudo code

Initialize $\mathbf{x}^{[0]}$, $t = 0$
while stopping criterion not met **do**
    Randomly shuffle indices and partition into minibatches $J_1, \dots, J_K$ of size m
    for $k \in \{1, \dots, K\}$ **do**
        $t \leftarrow t + 1$
        Compute gradient estimate with J_k: $\hat{\mathbf{d}}^{[t]} \leftarrow \frac{1}{m} \sum_{i \in J_k} \nabla_{\mathbf{x}} g_i(\mathbf{x}^{[t-1]})$
        Apply update: $\mathbf{x}^{[t]} \leftarrow \mathbf{x}^{[t-1]} - \alpha \cdot \hat{\mathbf{d}}^{[t]}$
    end for
end while

- Instead of drawing batches randomly we might want to go through the g_i sequentially (unless g_i are sorted in any way)
- Updates are computed faster, but also more stochastic:
  - In the simplest case, batch-size $m := |J_k|$ is set to $m = 1$
  - If n is a billion, computation of update is a billion times faster

SGD IN ML

- In ML, we perform ERM:

$$\mathcal{R}(\boldsymbol{\theta}) = \frac{1}{n} \sum_{i=1}^{n} \underbrace{L\left(y^{(i)}, f\left(\boldsymbol{x}^{(i)} \mid \boldsymbol{\theta}\right)\right)}_{g_i(\boldsymbol{\theta})}$$

- for a data set $\mathcal{D} = \left(\left(\boldsymbol{x}^{(1)}, y^{(1)}\right), \dots, \left(\boldsymbol{x}^{(n)}, y^{(n)}\right)\right)$
- a loss function $L\left(y, f(\boldsymbol{x})\right)$, e.g., L2 loss $L\left(y, f(\boldsymbol{x})\right) = (y - f(\boldsymbol{x}))^2$
- and a model class f, e.g., the linear model $f\left(\boldsymbol{x}^{(i)} \mid \boldsymbol{\theta}\right) = \boldsymbol{\theta}^\top \boldsymbol{x}$

SGD IN ML

- For large data sets, computing the exact gradient

$$\mathbf{d} = \frac{1}{n}\sum_{i=1}^{n} \nabla_{\boldsymbol{\theta}} L\left(y^{(i)}, f\left(\boldsymbol{x}^{(i)} \mid \boldsymbol{\theta}\right)\right)$$

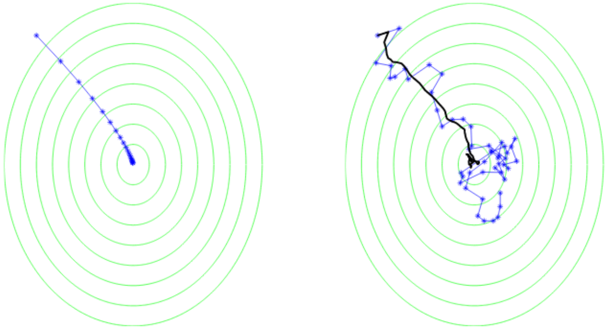
  may be expensive or even infeasible to compute and is approximated by

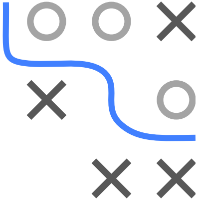
$$\hat{\mathbf{d}} = \frac{1}{m}\sum_{i\in J} \nabla_{\boldsymbol{\theta}} L\left(y^{(i)}, f\left(\boldsymbol{x}^{(i)} \mid \boldsymbol{\theta}\right)\right),$$

  for $J \subset \{1, 2, \ldots, n\}$ random subset

- **NB:** Often, maximum size of J technically limited by memory size

STOCHASTICITY OF SGD





▸ Click for source

Minimize $g(x_1, x_2) = 1.25(x_1 + 6)^2 + (x_2 - 8)^2$ **Left:** GD, **Right:** SGD
Black line shows average value across multiple runs
SGD trajectory much more noisy than of GD (due to stochastic gradient estimates)

STOCHASTICITY OF SGD

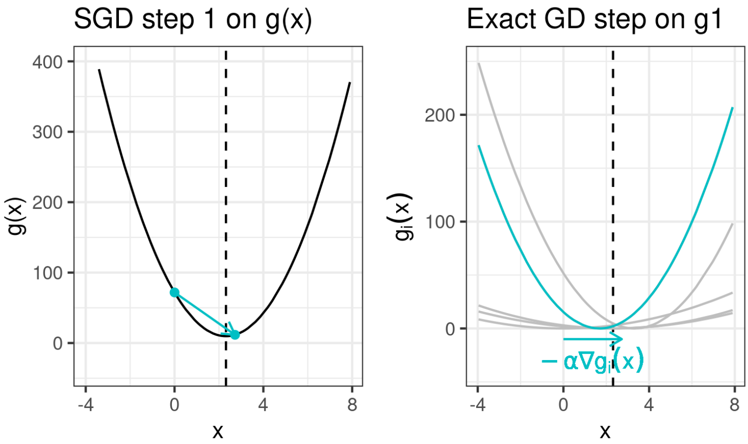
- Assume batch size $m = 1$ (statements also apply for larger batches)
- **(Possibly) suboptimal direction:** Approximate gradient $\hat{\mathbf{d}} = \nabla_{\boldsymbol{x}} g_i(\boldsymbol{x})$ might point in suboptimal (possibly not even a descent!) direction
- **Unbiased estimate:** If J drawn i.i.d., approximate gradient $\hat{\mathbf{d}}$ is an unbiased estimate of gradient $\mathbf{d} = \nabla_{\boldsymbol{x}} g(\boldsymbol{x}) = \frac{1}{n} \sum_{i=1}^{n} \nabla_{\boldsymbol{x}} g_i(\boldsymbol{x})$:

$$\begin{aligned}\mathbb{E}_i \left[\nabla_{\boldsymbol{x}} g_i(\boldsymbol{x})\right] &= \sum_{i=1}^{n} \nabla_{\boldsymbol{x}} g_i(\boldsymbol{x}) \cdot \mathbb{P}(i = i) \\ &= \sum_{i=1}^{n} \nabla_{\boldsymbol{x}} g_i(\boldsymbol{x}) \cdot \frac{1}{n} = \nabla_{\boldsymbol{x}} g(\boldsymbol{x}).\end{aligned}$$

- **Conclusion:** SGD might perform single suboptimal moves, but moves in “right direction” **on average**

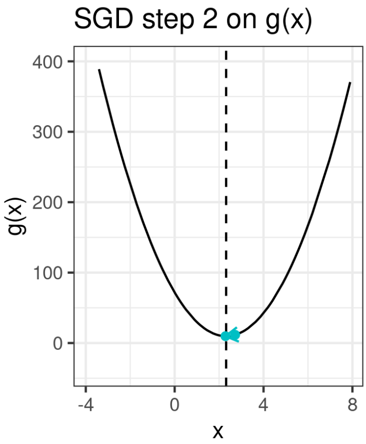
ERRATIC BEHAVIOR OF SGD

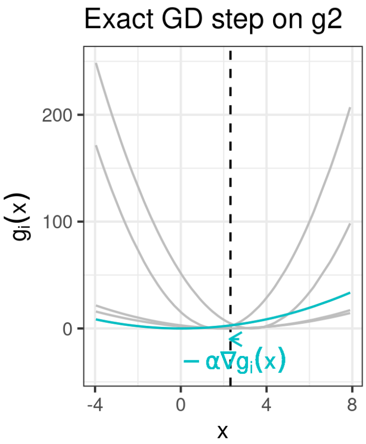
Example: $g(\boldsymbol{x}) = \sum_{i=1}^{5} g_i(\boldsymbol{x})$, g_i quadratic, batch size $m = 1$
(Note that each SGD step for $m = 1$ corresponds to an exact GD step on one of the g_i.)



ERRATIC BEHAVIOR OF SGD

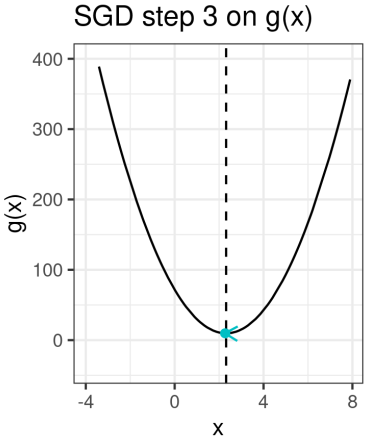
Example: $g(\boldsymbol{x}) = \sum_{i=1}^{5} g_i(\boldsymbol{x})$, g_i quadratic, batch size $m = 1$

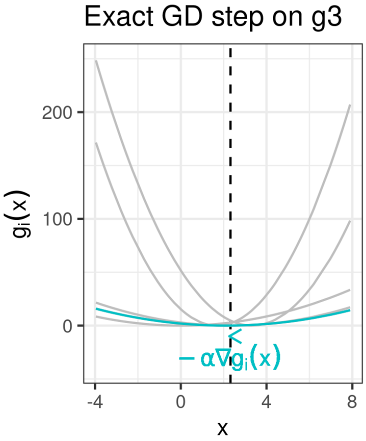


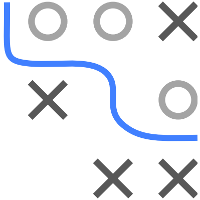


ERRATIC BEHAVIOR OF SGD

Example: $g(\boldsymbol{x}) = \sum_{i=1}^{5} g_i(\boldsymbol{x})$, g_i quadratic, batch size $m = 1$

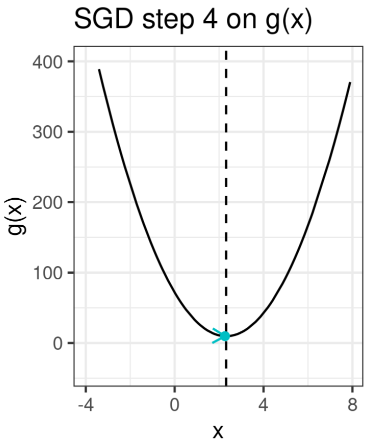


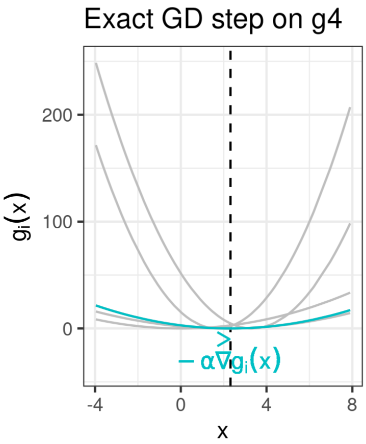


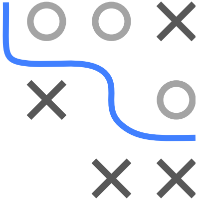


ERRATIC BEHAVIOR OF SGD

Example: $g(\mathbf{x}) = \sum_{i=1}^{5} g_i(\mathbf{x})$, g_i quadratic, batch size $m = 1$

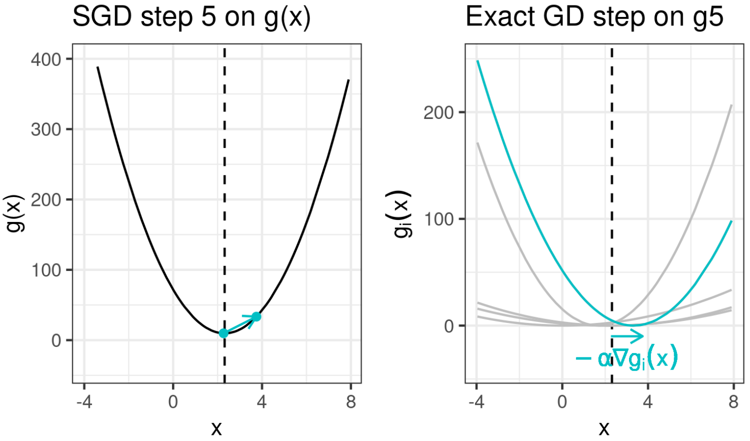




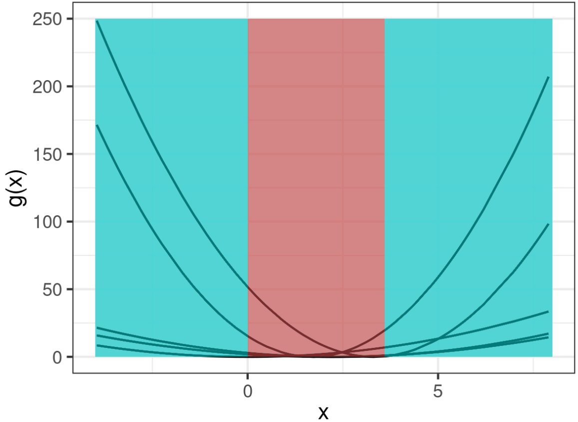


ERRATIC BEHAVIOR OF SGD

Example: $g(\mathbf{x}) = \sum_{i=1}^{5} g_i(\mathbf{x})$, g_i quadratic, batch size $m = 1$
In iteration 5, SGD performs a suboptimal move away from the minimum:



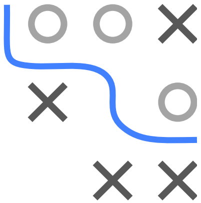
ERRATIC BEHAVIOR OF SGD



Blue area: Each $-\nabla g_i(\boldsymbol{x})$ points towards minimum of $g(\boldsymbol{x})$

Red area ("confusion area"): $-\nabla g_i(\boldsymbol{x})$ might point away from minimum and perform a suboptimal move

ERRATIC BEHAVIOR OF SGD



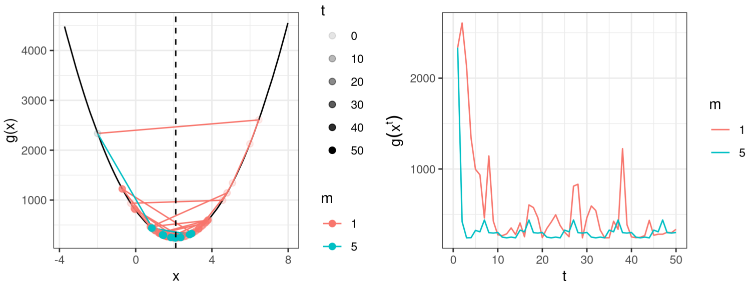
- At location $\boldsymbol{x}$, "confusion" is captured by **variance of gradients**

$$\frac{1}{n}\sum_{i=1}^{n}\|\nabla_{\boldsymbol{x}} g_i(\boldsymbol{x}) - \nabla_{\boldsymbol{x}} g(\boldsymbol{x})\|^2$$

  - If term is 0, next step goes in gradient direction (for each i)
  - If term is small, next step *likely* goes in gradient direction
  - If term is large, next step likely goes in direction different than gradient
- Consequently, SGD has worse convergence properties than GD
- **But:** Can be controlled by tuning **batch size** and **step size**.

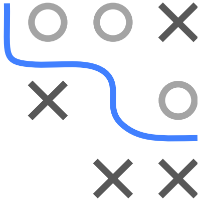
EFFECT OF BATCH SIZE

- The larger the batch size m
  - the better the approximation to $\nabla_{\boldsymbol{x}} g(\boldsymbol{x})$
  - the lower the risk of performing steps in the wrong direction
  - the lower the variance
  - **But:** Maximum batch size is usually limited by computational resources (memory)



Example continued: Batch size $m = 1$ vs. $m = 5$

EFFECT OF STEP SIZE



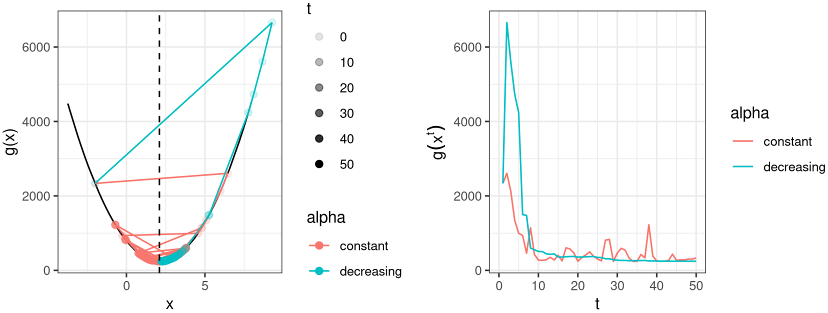
- The smaller the step size α
  - the smaller erratic steps in potentially wrong directions
  - the lower the effect of gradient variance
  - **But:** Risk of impaired performance by not moving far enough
- SGD converges for sufficiently smooth functions if

$$\frac{\sum_{t=1}^{\infty} \left(\alpha^{[t]}\right)^2}{\sum_{t=1}^{\infty} \alpha^{[t]}} = 0$$

(“how much noise affects you” by “how far you can get”)

STEP SIZE CONTROL FOR SGD

- Popular solution: Decreasing step size
  - Initially, step size shall be large enough to make progress
  - In later iterations, step size shall be small enough to reduce effect of gradient variance



Example continued: Step size $\alpha^{[t]} = 0.2/t$

Decreasing step size leads to more stable convergence

STEP SIZE CONTROL FOR SGD

Why not Armijo-based step size control?

- Backtracking line search or other approaches based on Armijo rule usually not suitable: Armijo condition

$$g(\boldsymbol{x} + \alpha \mathbf{d}) \leq g(\boldsymbol{x}) + \gamma_1 \alpha \nabla g(\boldsymbol{x})^\top \mathbf{d}$$

  requires evaluating full gradient
- But SGD is used to *avoid* expensive gradient computations
- Research aims at finding inexact line search methods that provide better convergence behavior, e.g., ▸ Vaswani et al. 2019

STOPPING RULES FOR SGD

- For GD: We usually stop when gradient is close to 0 (i.e., we are close to a stationary point)
- For SGD: Individual gradients do not necessarily go to zero, and we cannot access full gradient
- Practicable solution for ML:
  - Measure the validation set error after T iterations
  - Stop if validation set error is not improving

SGD AND ML

General remarks:

- SGD is a variant of GD
- SGD particularly suitable for large-scale ML when evaluating gradient is too expensive
- Thus, SGD and variants are the most commonly used methods in modern ML with e.g. neural networks
- Note that even for the linear model and quadratic loss, where a closed form solution is available, SGD is often used if the size n of the dataset is too large and the design matrix does not fit into memory